

CalendarView 周视图日历实现

CalendarView.swift 日历视图实现详解

CalendarView.swift 是项目中的周视图日历组件，专注于“周维度任务展示与管理”，下文从核心功能、数据获取、界面刷新三方面拆解其实现逻辑。

一、核心功能

日历视图围绕“周任务管理”设计，覆盖“查看、操作、导出”全流程，核心功能如下：

- 周维度展示**：横向排列“周一至周日”，每个日期单元格显示当天任务数量及简要信息（如任务标题、状态）；
- 周切换交互**：支持左右滑动切换“上一周 / 下一周”，也可通过顶部“周数按钮”快速跳转；
- 任务详情操作**：点击任务条目，弹出详情弹窗，支持查看任务完整信息（如截止时间、描述）及编辑修改；
- 任务拖拽排序**：长按任务条目，可拖拽调整任务在当天列表中的显示顺序；
- PDF 导出**：支持将当前周的日历与任务列表导出为 PDF 文件，可分享至邮件、备忘录等应用；
- 集成聊天输入框**：底部内置聊天输入组件，支持快速发送任务相关消息（如任务协作沟通）。

二、数据获取逻辑

数据获取通过 `loadTasks()` 方法实现，采用“缓存优先，网络兜底”策略，确保离线可用、在线更新，流程如下：

1. 第一步：优先加载缓存数据（离线可用）

- 缓存存储位置**：数据缓存至 `UserDefaults`，避免频繁网络请求；
- 缓存键规则**：缓存键由“用户 ID + 年份 + 周数”组成（如 `userId_2024_20`），确保不同用户、不同周的数据唯一；
- 加载逻辑**：视图初始化时，先根据当前周的缓存键查询 `UserDefaults`，若存在缓存数据，立即解析并显示，减少用户等待时间。

2. 第二步：网络请求更新数据（在线同步）

- **加载状态标记**：网络请求前，将 `isLoadingFromServer`（加载状态）设为 `true`，显示加载指示器（如转圈圈）；
- **网络请求**：调用 `TaskService` 的 `getTasksByWeek(weekNum, year)` 方法，从服务器获取当前周的最新任务；
- **结果处理**：
 - 成功：更新本地任务数据（`tasks`），同时将新数据缓存至 `UserDefaults`，覆盖旧缓存；
 - 失败：显示错误提示（如“获取任务失败，请重试”），3 秒后自动隐藏，不阻断用户操作。

3. 数据更新触发点（何时加载数据）

- 视图首次加载：通过 `task` 修饰符，视图出现时自动调用 `loadTasks()`；
- 周视图切换：通过 `onChange` 监听 `currentWeek`（当前周）变化，切换周时重新加载对应周数据；
- 下拉刷新：通过 `refreshable` 修饰符，用户下拉列表时触发 `loadTasks()`，强制更新当前周数据；
- 任务编辑后：通过通知中心监听“任务编辑完成”通知（如 `Notification.Name("TaskEdited")`），收到通知后调用 `loadTasks()`，同步最新任务状态。

三、界面刷新逻辑

界面刷新依赖 SwiftUI 响应式特性，结合“状态驱动 + 通知监听”，实现数据变化后界面自动更新，无需手动操作。

1. 状态驱动刷新（核心机制）

- **状态管理**：使用 `@State/@StateObject` 管理关键状态（如 `tasks` 任务列表、`currentWeek` 当前周、`isLoadingFromServer` 加载状态）；
- **自动渲染**：当状态变化时（如 `tasks` 新增 / 删除任务、`isLoadingFromServer` 切换加载状态），- SwiftUI 自动重新绘制依赖该状态的 UI 元素（如任务列表刷新、加载指示器显示 / 隐藏）。

2. 通知监听刷新（跨组件同步）

- **通知注册**：视图初始化时，通过 `NotificationCenter` 注册 “刷新任务” 通知（如 `Notification.Name("RefreshTasksInCalendar")`）；
- **通知响应**：收到通知后，调用 `refreshTasks()` 方法，触发强制刷新；
- **适用场景**：其他组件（如任务编辑页）修改任务后，通过发送通知，触发日历视图同步更新，避免组件间直接依赖。

3. 强制刷新逻辑（`refreshTasks()` 方法）

```
private func refreshTasks() async {  
    // 1. 清除当前周的本地缓存（强制从网络获取最新数据）  
    let cacheKey = taskCacheKeyForUser(suffix: "\(year)_\(currentWeek)")  
    UserDefaults.standard.removeObject(forKey: cacheKey)  
  
    // 2. 重新加载数据（网络请求优先）  
    await loadTasks()  
}
```

4. 滑动切换动画（周切换流畅性）

- **手势识别**：通过 `DragGesture` 识别用户左右滑动手势，获取滑动距离与速度；
- **切换判断**：设定滑动阈值（如屏幕宽度的 30%），滑动距离超过阈值时触发周切换；
- **过渡动画**：
 - 向右滑动（切换上一周）：当前周视图向右滑出，上一周视图从左侧滑入，使用 `withAnimation-` 包裹状态变化，添加平滑过渡；
 - 向左滑动（切换下一周）：当前周视图向左滑出，下一周视图从右侧滑入，动画逻辑与向右滑动对称；
- **代码示例**（滑动手势处理）：

```
.gesture(  
    DragGesture()  
    .onChanged { value in  
        isDragging = true // 标记正在滑动，避免重复触发  
        let rawOffset = value.translation.width  
        dragOffset = rawOffset * 0.8 // 增加滑动阻力感，提升交互体验  
    }
```

```

.onEnded { value in
    isDragging = false
    let screenWidth = geometry.size.width
    let threshold = screenWidth * 0.3 // 滑动阈值：屏幕宽度的30%

    // 向右滑动：切换上一周
    if value.translation.width > threshold {
        withAnimation(.easeInOut(duration: 0.3)) {
            dragOffset = screenWidth // 当前视图向右滑出
        }
        // 延迟执行周切换，等待动画完成
        DispatchQueue.main.asyncAfter(deadline: .now() + 0.3) {
            currentWeek -= 1 // 切换到上一周
            dragOffset = 0 // 重置偏移量
            Task { await loadTasks() } // 加载上一周数据
        }
    }
    // 向左滑动：切换下一周
    else if value.translation.width < -threshold {
        withAnimation(.easeInOut(duration: 0.3)) {
            dragOffset = -screenWidth // 当前视图向左滑出
        }
        DispatchQueue.main.asyncAfter(deadline: .now() + 0.3) {
            currentWeek += 1 // 切换到下一周
            dragOffset = 0
            Task { await loadTasks() } // 加载下一周数据
        }
    }
    // 滑动未达阈值：重置视图位置
    else {
        withAnimation(.easeInOut(duration: 0.2)) {
            dragOffset = 0
        }
    }
}
)

```

四、关键代码示例

1. 数据加载核心代码 (loadTasks() 方法)

```
private func loadTasks() async {
    // 1. 构建缓存键 (用户ID + 年份 + 周数)
    guard let userId = UserDefaults.standard.string(forKey: "currentUserId") else { return }
    let suffix = "\(year)_\(currentWeek)"
    let cacheKey = "\(userId)_weekTasks_\(suffix)"

    // 2. 加载缓存数据
    if let cachedData = UserDefaults.standard.data(forKey: cacheKey),
        let cachedTasks = try? JSONDecoder().decode([Task].self, from: cachedData) {
        self.tasks = cachedTasks
        self.previewTasks = cachedTasks // 预览数据同步更新
    }

    // 3. 网络请求更新数据
    isLoadingFromServer = true
    do {
        // 调用TaskService获取当前周任务
        let serverTasks = try await TaskService.shared.getTasksByWeek(week: currentWeek, year:
year)
        // 排序任务 (按截止时间升序)
        let sortedTasks = TaskService.shared.sortTasksByDeadline(tasks: serverTasks)
        // 更新本地数据
        self.tasks = sortedTasks
        self.previewTasks = sortedTasks
        // 缓存新数据 (覆盖旧缓存)
        let encodedData = try JSONEncoder().encode(sortedTasks)
        UserDefaults.standard.set(encodedData, forKey: cacheKey)
    } catch {
        // 显示错误提示 (3秒后自动隐藏)
        showError = true
        DispatchQueue.main.asyncAfter(deadline: .now() + 3) {
            showError = false
        }
        print("获取周任务失败: \(error.localizedDescription)")
    }
    // 结束加载状态
    isLoadingFromServer = false
}
```

总结

CalendarView.swift 基于 SwiftUI 响应式特性，实现了 “功能完整、交互流畅、性能优化” 的周视图日历：

- **功能层面**：覆盖周任务展示、切换、编辑、导出，满足日常任务管理需求；
- **数据层面**：“缓存优先 + 网络兜底” 策略，平衡离线可用性与数据新鲜度；
- **体验层面**：滑动切换动画、加载状态反馈、错误提示自动隐藏，提升用户操作流畅性；
- **扩展性层面**：通过通知监听实现跨组件同步，便于后续集成更多功能（如团队任务共享）。

（注：文档部分内容可能由 AI 生成）